

SIH26045 — IP-SAKTI Sahayak

Technical Execution Guide

PS Number: SIH26045 **Title:** IP-SAKTI Sahayak — a multilingual, RAG-based (source-cited) AI assistant for Intellectual Property and regulatory guidance in Ayurveda **Ministry:** Ministry of Ayush **Category:** Software

1. What We're Building

An AI assistant that answers questions about Ayurveda-related IP and regulatory rules by retrieving text from official government documents and showing exactly which document and section each part of the answer came from.

Scope for the prototype: National (Indian) framework only. International IP regimes go into the “future work” slide.

1.5 Timeline — Internal Hackathon: 6th September (Online)

We have ~8 days. This changes what's “core path” vs. “stretch goal.” Everything below is triaged against that deadline.

Core path (must ship): - Module 1 — Data Collection - Module 2 — Ingestion Pipeline - Module 2.5 — Query Rewriter - Module 3 — Hybrid Retrieval - Module 5 — Grounded Generation - Module 6 — Citation Attachment (the “attach” step is mandatory; the sentence-level “verify” step is a stretch add if time allows) - Module 7 — LangGraph Wiring (sequential DAG only — skip the optional conditional retry branch) - Module 9 — API Layer (minimal: /query endpoint is enough) - Module 10 — Frontend (minimal: chat input + answer + sources list; polish later) - Module 12 — Demo Prep

Stretch (only if core path is done early): - Module 4 — Reranking - Module 6.5 — Evaluation Set (scale down to 8-10 verified Q&A pairs instead of 20-30 if attempted at all) - Module 8 — Multilingual Layer - Module 11 — Full production deployment (a local/localhost demo is acceptable for an online internal round; deploy only if time allows)

Suggested day-by-day (adjust as the team sees fit):

Day	Focus
Day 1 (today)	Team roles finalized, repo created, Module 1 (data collection) starts immediately
Day 2	Module 2 (ingestion) built and verified; Module 10 frontend scaffolding starts against mock data in parallel
Day 3	Module 2.5 (query rewriter) + Module 3 (hybrid retrieval)
Day 4	Module 5 (grounded generation) + start Module 6 (citation attachment) Finish Module 6; Module 7

Day 5	(LangGraph wiring, sequential only); integration testing begins
Day 6	Module 9 (API layer) connected to real frontend; end-to-end testing
Day 7	Buffer day — fix whatever broke in end-to-end testing; only attempt stretch items if core path is fully stable
Day 8 (6th Sept)	Module 12 — demo rehearsal, backup video recorded, PPT finalized; online demo dry run on the actual platform used for judging

Daily check-in is mandatory, not optional, given this window — a short standup (10 min, async on WhatsApp if a call isn't possible) so blockers surface same-day, not two days later.

2. Why Our Architecture Is Different (the winning angle)

Most teams will build a basic RAG chatbot: embed the query, fetch top-k chunks, feed to an LLM, done. That approach has known, measurable problems. A peer-reviewed Stanford study (Magesh, Surani, Dahl, Suzgun, Manning & Ho, "Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools," *Journal of Empirical Legal Studies*, 2025) found that leading legal AI research tools — LexisNexis's Lexis+ AI and Thomson Reuters's Westlaw AI-Assisted Research and Ask Practical Law AI — hallucinated **17-33% of the time**, despite provider claims that retrieval-augmented generation eliminates hallucinations. Lexis+ AI's answers were fully accurate for only 65% of queries; Westlaw for 41%; Practical Law AI for just 19%. In a regulatory domain, a confidently wrong answer is worse than no answer.

(This citation is verified — safe to state precisely in the PPT and to judges, and to name the study directly if asked "where's that from.")

We are building four things that basic RAG does not have:

1. Hybrid retrieval instead of dense-only. Benchmarks consistently show that BM25 (keyword matching) combined with dense embeddings (semantic matching), fused via Reciprocal Rank Fusion, outperforms either method alone. This matters especially in legal text where exact terms and section numbers must match, not just "similar meaning."

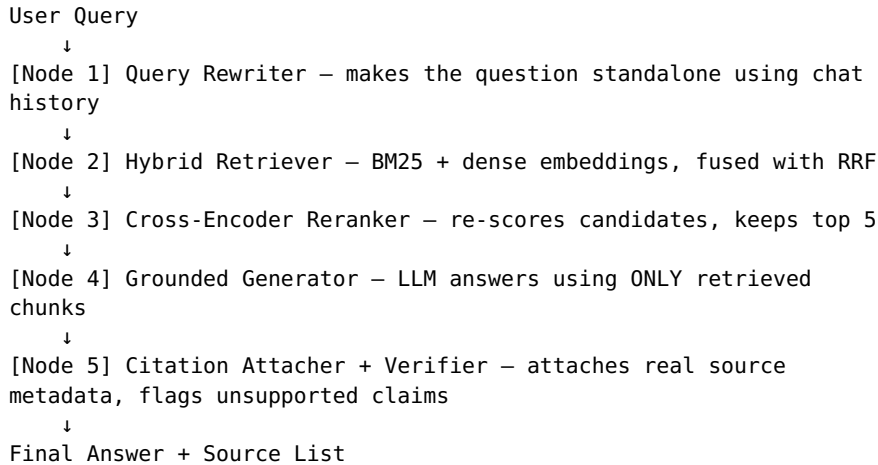
2. Cross-encoder reranking. After retrieving a wide set of candidates, a reranker model re-scores them for actual relevance to the question. This meaningfully improves the quality of what reaches the LLM.

3. Programmatic citations, not LLM-generated citations. The single most important design decision. LLMs fabricate plausible-looking references — this is a documented failure mode. So the LLM never writes the citation. Our code tracks which chunks were retrieved (with their source filename and section metadata) and attaches those to the answer. The citation is a fact about our retrieval, not something the model invented.

4. Abstention guardrail. If the retrieved context doesn't contain the answer, the system says "I don't have this in my sources" and asks a clarifying question, instead of guessing. This also protects us during the live demo if a judge asks something outside our document set.

Orchestration choice: We use **LangGraph as a deterministic DAG**, not as a fully autonomous agent loop. LangGraph is the most mature orchestration framework for this, but fully autonomous agentic loops are hard to debug, non-deterministic, and produce the same answer as simpler pipelines on straightforward factual questions at several times the cost. A fixed graph gives us the architectural sophistication without the debugging risk — and it’s easy to add a conditional branch later if time allows.

3. System Architecture



Each node is a LangGraph node with typed state passing between them. The whole graph is deterministic and traceable — we can show judges exactly what happened at each step.

4. Tech Stack

Layer	Tool	Why this one
Orchestration	LangGraph (deterministic DAG mode)	Mature, traceable, impressive to judges, debuggable
Backend framework	FastAPI (Python)	Fast, async, easy deploy
Sparse retrieval	rank_bm25 (Python library)	Free, exact keyword/section matching
Dense retrieval	sentence-transformers embeddings	Free, runs locally, no API cost
Vector store	pgvector (Postgres extension)	Free, reliable, well-supported
Reranker	Cross-encoder (ms-marco-MiniLM class model)	Free, local, big accuracy lift
LLM — primary	Local model via Ollama (Llama 3 / Mistral, quantized)	Works offline — demo-safe
LLM — fallback	Groq API (free tier)	Better quality when internet available
Translation	Bhashini API (Govt of India) or Google Translate free tier	Hindi support; Bhashini adds govt alignment
Frontend	React / Next.js	Fast to build a clean chat UI

Backend hosting	Render or Railway (free tier)	Easy CI/CD from GitHub
Frontend hosting	Vercel (free tier)	One-click deploy
DB hosting	Supabase or Neon (free tier, pgvector supported)	Managed, no server admin
Version control	GitHub (private repo)	Team collaboration

5. Repository Structure

```

ip-sakti/
├── backend/
│   ├── ingestion/
│   │   ├── loader.py          # PDF → raw text
│   │   ├── chunker.py        # text → chunks with metadata
│   │   └── indexer.py         # chunks → BM25 index + pgvector
│   ├── graph/
│   │   ├── state.py          # LangGraph state schema
│   │   ├── nodes.py          # the 5 nodes
│   │   └── build_graph.py     # wires nodes into the DAG
│   ├── retrieval/
│   │   ├── query_rewriter.py # standalone-query rewriting
│   │   ├── bm25_search.py
│   │   ├── dense_search.py
│   │   ├── fusion.py          # Reciprocal Rank Fusion
│   │   └── reranker.py
│   ├── generation/
│   │   ├── prompts.py        # citation-enforced system prompts
│   │   ├── llm_client.py     # Ollama primary + Groq fallback
│   │   └── citation.py        # programmatic citation attachment
│   ├── api/
│   │   └── main.py            # FastAPI endpoints
│   └── requirements.txt
├── frontend/
│   └── (React/Next.js chat UI)
├── data/
│   └── (source PDFs)
├── eval/
│   ├── test_set.json          # 20-30 verified Q&A pairs with
│   │   expected citations
│   └── run_eval.py            # scores the pipeline against
│       test_set.json
└── docs/
    └── (architecture notes, PPT assets)

```

6. Build Steps — Module by Module

Module 1: Data Collection

What to do: Download 10-15 official documents.

Where from: - ayush.gov.in — AYUSH ministry circulars on IP and traditional knowledge protection - ipindia.gov.in — Indian Patent Office guidelines on Ayurveda/traditional medicine patents - GI Registry — Geographical Indication documents for Ayurvedic products - ctri.nic.in — Clinical Trials Registry India registration guidelines - NDCT Rules 2019 — relevant sections - TKDL (Traditional Knowledge Digital Library) related guidelines

How: Save as PDFs into /data. Name files clearly (e.g. AYUSH_IP_Circular_2023.pdf) — the filename becomes part of the citation shown to users, so it must be readable.

Note: This step needs no laptop — can be done from a phone browser.

Module 2: Ingestion Pipeline

What to build: A script that turns PDFs into a searchable index.

Steps: 1. **Load** — extract text from each PDF (PyMuPDF or pdfplumber; pdfplumber handles tables better) 2. **Chunk** — split into ~500-token chunks with ~50-token overlap so context isn't cut mid-sentence 3. **Attach metadata to every chunk** — this is critical: - source_file (document name) - page_number - section_heading (if detectable) - chunk_id 4. **Index twice:** - Build a BM25 index over the chunk texts - Generate embeddings and store chunk + embedding + metadata in pgvector

How to verify it worked: Query the database directly, confirm each row has non-empty metadata. If source_file is missing anywhere, citations will break later.

Module 2.5: Query Rewriter (*new*)

What to build: A node that takes the current question plus recent chat history and rewrites it into a standalone question if it depends on prior context.

Example: “What about GI?” → “What are the GI registration requirements for Ayurvedic products?”

How: A single LLM call with a short prompt: *“Given this conversation and the latest question, rewrite the latest question to be fully self-contained. If it's already self-contained, return it unchanged.”*

How to verify: Run a 3-turn conversation where the last question uses a pronoun or implicit reference. Confirm the rewritten query makes sense standalone before it hits retrieval.

Module 3: Hybrid Retrieval

What to build: Two retrievers plus a fusion step.

Steps: 1. **BM25 search** — takes the query, returns top 20 chunks ranked by keyword overlap 2. **Dense search** — embeds the query, returns top 20 chunks by cosine similarity from pgvector 3. **Reciprocal Rank Fusion** — merge both ranked lists into one. RRF scores each document by summing $1 / (k + \text{rank})$ across the lists it appears in (k is typically 60). Documents that rank well in both methods rise to the top. 4. Return the fused top ~20 candidates

Why both: Dense search finds “what does this mean” matches; BM25 finds “this exact section number / legal term” matches. Regulatory queries need both.

How to verify: Test with a query containing a specific term (e.g. a rule number). BM25 should surface the exact document. Then test a vague conceptual query — dense should surface the right topic. Confirm fusion returns sensible results for both.

Module 4: Reranking

What to build: A cross-encoder pass over the 20 fused candidates.

Steps: 1. Load a cross-encoder model (runs locally, free) 2. Score each (query, chunk) pair — cross-encoders read both together, which is more accurate than comparing separate embeddings 3. Keep the top 5 by score 4. Pass those to generation

How to verify: Compare top-5 before and after reranking on a few test queries. The reranked set should be visibly more on-topic. This before/after comparison is also good demo material.

Module 5: Grounded Generation

What to build: The LLM call, with strict constraints.

System prompt must enforce: - Answer using ONLY the provided context chunks - Do not use outside knowledge, even if you're confident - If the context doesn't contain the answer, say so explicitly and ask one clarifying question - Do not write citations yourself — just answer; the system handles attribution

LLM setup: - Primary: Ollama running a quantized local model - Fallback: Groq API, triggered if the local model is unavailable or times out - Both use the identical prompt so behaviour stays consistent

How to verify: Ask a question you know is NOT in the documents. The system must abstain, not invent an answer. This is a core test — run it before every demo.

Module 6: Citation Attachment & Verification

What to build: The trust layer.

Steps: 1. **Attach:** After generation, take the metadata from the chunks that were actually retrieved and attach them as the source list. The LLM had no input into this — it's derived from our retrieval records, so it cannot be fabricated. 2. **Verify (optional but strong):** Split the generated answer into sentences. For each sentence, check similarity against the retrieved chunks. Flag any sentence with low similarity as "unsupported." Either hide flagged sentences or mark them visually.

Why this wins points: It directly addresses the documented weakness of legal AI systems. Being able to tell judges "our citations cannot be hallucinated because the model never generates them" is a strong, specific, defensible claim.

Module 6.5: Evaluation Set (new)

What to build: A held-out test set and scoring script, so accuracy is a number we can state, not a claim we assert.

Why this matters: The PS explicitly lists "answer accuracy, citation correctness, safe abstention" as evaluation criteria. "We tested it and it seemed to work" is not an answer judges will find convincing — a score is.

Steps: 1. Create `eval/test_set.json` with 20–30 question-answer pairs. Each entry has: the question, the correct answer (written in our own words, verified against the source — see Domain Accuracy Owner below), and the exact document + section it should cite. 2. Include a mix: straightforward factual questions, a few questions that require the jurisdiction/classification logic, and 3–5 deliberately out-of-scope questions to test abstention. 3. Write `eval/run_eval.py` to run every question through the full pipeline and check: (a) did it retrieve the right source document, (b) is the cited section correct, (c) does the answer match the expected answer semantically, (d) did it correctly

abstain on out-of-scope questions. 4. Track and report a simple score, e.g. “22/25 correctly cited, 4/5 correct abstentions.” Put this number in the PPT.

When to start: As soon as Module 2 (ingestion) is stable — this should run continuously as the pipeline develops, not get built last-minute the night before the demo.

Module 7: LangGraph Wiring

What to build: Connect Modules 2.5–6 as graph nodes.

Steps: 1. Define a state object holding: query, rewritten_query, candidates, reranked, answer, citations, flags 2. Create one node per stage — each takes state, does its job, returns updated state 3. Wire them in sequence with add_edge 4. Compile the graph

Optional conditional branch (only if time allows): After reranking, if the top score is below a threshold, route to a query-rewrite node and retry retrieval once. This adds a genuine agentic element without the risk of an unbounded loop.

How to verify: Enable LangGraph tracing and run one query end to end. Confirm state flows correctly through every node.

Module 8: Multilingual Layer

What to build: Hindi in, Hindi out.

Steps: 1. Detect query language 2. If Hindi: translate to English before it enters the graph 3. Run the graph normally (documents are in English) 4. Translate the final answer back to Hindi — but keep source document names untranslated, since they’re official titles

Priority note: This is valuable but not load-bearing. If time is tight, ship English-only and present Hindi support as in-progress. Do not let this block Modules 2–7.

Module 9: API Layer

What to build: FastAPI endpoints.

Endpoints needed: - POST /query — accepts a question (plus optional chat history), returns answer + citations + flags - GET /health — for the hosting platform’s health checks - POST /ingest — optional admin endpoint to add documents

Also handle: CORS for the frontend, request timeouts, and a clear error response if the LLM is unreachable.

Module 10: Frontend

What to build: A clean chat interface.

Must have: - Message input and conversation history - Each answer displays a **Sources** section — document name, page/section, and ideally the exact retrieved snippet in an expandable view - A visible indicator when the system abstains (“not found in sources”) - Loading state during retrieval

Design principle: The citation display is the product’s differentiator — make it prominent, not a footnote. Judges should see the traceability without being told about it.

Module 11: Deployment

Steps: 1. Backend → Render or Railway, connected to the GitHub repo 2. Frontend → Vercel, connected to the same repo 3. Database → Supabase or Neon with the pgvector extension enabled 4. Set environment variables (DB URL, Groq key) in each platform's dashboard — never commit keys 5. Run the ingestion script once against the production database 6. Test the deployed URLs end to end, not just localhost

Important: Hosted free-tier instances may not run a local LLM. Plan for the deployed version to use the Groq fallback, and keep the local Ollama setup for the offline demo path.

Module 12: Demo Preparation

Note: internal round is online (6th September). Adjust prep accordingly — the offline-WiFi contingency matters less than platform reliability and screen-share logistics.

Steps: 1. Prepare 5–6 questions you've verified the system answers well (pull these from the eval set if built, or from manual testing if Module 6.5 was skipped for time) 2. Prepare one question you know is out of scope — demo the abstention. This is a feature, not a weakness, and showing it deliberately signals engineering maturity. 3. Confirm the exact demo format with organizers: live code walkthrough, live product demo, pre-recorded video submission, or pitch + live Q&A — this changes how much UI polish vs. rehearsed talking points matter. 4. Do a full dry run **on the actual platform being used for judging** (Zoom/Meet/whatever the college specifies) — test screen share, audio, and app performance under that specific setup, not just localhost. 5. Record a backup demo video regardless of format — online demos still fail from local crashes, laggy models, or a bad screen-share moment. 6. Build the PPT: problem → why existing approaches fail (cite the verified Stanford 17–33% hallucination finding) → our architecture → eval score (if available) → live demo → impact → future work (international regimes, more documents, professional legal review, deeper verification)

7. Build Order & Parallelisation

See Section 1.5 for the actual day-by-day plan against the 6th September deadline. The dependency logic below still holds — it just needs to be compressed into 8 days, with Modules 4, 6.5, 8, and 11 treated as stretch goals rather than sequential requirements.

Sequential dependency chain: **Module 1 → 2 → 2.5 → 3 → (4) → 5 → 6 → (6.5) → 7 → 9 → (11) → 12** (*parenthesized modules are stretch goals — see Section 1.5*)

Can run in parallel: - Module 1 (data collection) — anyone, needs no laptop - Module 10 (frontend) — can be built against mock API responses while backend is in progress - Module 6.5 (eval set) — can start being drafted as soon as Module 1's documents are chosen, verified in parallel with Modules 3–6 being built - Module 12 (PPT) — can be drafted from this document while code is being built - Module 8 (multilingual) — bolt on after Module 7 works; skippable

8. Team Roles (*rebalanced — no single point of failure*)

Role	Modules
------	---------

Retrieval Lead	2, 2.5, 3, 4
Generation/Graph Lead	5, 6, 7
Frontend Lead	10
Data + Eval Lead	1, 6.5
Integration/Multilingual Lead	8, 9, 11
Presentation Lead	12
Domain Accuracy Owner (<i>new — can overlap with another role</i>)	Manually verifies every entry in eval/test_set.json against the actual source documents before the eval script is trusted; spot-checks citations for legal correctness, not just retrieval correctness

Note: the original plan concentrated Modules 2-7 in one person. If team size is tight, the Retrieval Lead and Generation/Graph Lead roles can be held by two people who split before/after the LangGraph wiring point rather than one person owning the entire core pipeline — this avoids a single point of failure blocking the whole build.

9. Key Talking Points for Judges

1. **We do not use dense-only retrieval** — hybrid BM25 + dense with RRF fusion, because exact legal terms and section numbers matter in this domain.
2. **Our citations cannot be hallucinated** — the model never generates them; they come from our retrieval records.
3. **The system abstains** rather than guessing when the answer isn't in the sources.
4. **It runs fully offline** — local model as primary, cloud API as an enhancement, not a dependency.
5. **Deterministic, traceable graph architecture** — we can show exactly what happened at every step of any query.
6. **We measured our accuracy, not just claimed it** — [X/Y] correct citations and [X/Y] correct abstentions on our held-out evaluation set.
7. **We know the failure mode we're solving** — a peer-reviewed Stanford study found leading legal AI tools hallucinate 17-33% of the time even with RAG in place; our architecture is built specifically around preventing the two causes that study identified as most common (LLM-generated citations and ungrounded generation).

10. What To Do First

1. Create the private GitHub repo with the structure in Section 5, add all teammates
2. Assign roles from Section 8, including the Domain Accuracy Owner
3. Start Module 1 immediately — no laptop required
4. Whoever has a laptop: set up the Postgres + pgvector instance and start Module 2
5. Frontend Lead: start Module 10 against mock responses in parallel
6. Daily check-in on who is blocked — non-negotiable given the compressed timeline (see Section 1.5)